

MODULE 9

Optimisation

Quantisation & pruning: same model, fewer bytes, fewer cycles

TinyML Course · Day 3

Why optimise? Our budget, honestly

nRF52840: **256 KB RAM · 1 MB flash · 64 MHz Cortex-M4F**

Already living there: Arduino BSP + SoftDevice, serial stack, your DSP code, double buffers, the EI SDK...

- **Memory:** CNN activations + float32 weights eat RAM fast
- **Speed:** shorter inference → higher sample duty cycle, or more sleep
- **Energy:** sleep is the whole business case — compute beats transmit by ~50× (cattle-collar case, Module 6)

Optimisation is not polish. On batteries, it is the product.

The four dials (Day 1 recap)

Model size	Memory	Compute	Performance
# parameters	peak activations	MACs / FLOPs	latency (ms)
model bytes	RAM at inference	ops count	energy / inference
float32 / int8			inferences / s

- **Quantisation** → shrinks *bytes per value* (weights **and** activations)
- **Pruning** → shrinks *number of values* (parameters, and sometimes MACs)

Model size = #params × bytes/param

Format	Bytes/param	1 M params	Our CNN (~5 k params)
float32	4	4 MB	~19 kB
float16	2	2 MB	~10 kB
int8	1	1 MB	~5 kB

- Dense fan NN (~1 k params): fits anywhere — flash is not its problem
- But: EI deployment = model + **DSP code + SDK**; and RAM, not flash, is usually the wall

RAM is the scarcer resource

- **Weights** → **flash** (persistent, 1 MB, usually fine)
- **Activations** → **RAM** (volatile, 256 KB, contested)
- Peak activation = largest intermediate tensor during inference

Example (Day 1): $28 \times 28 \times 1 \rightarrow \text{Conv}(8) \rightarrow 28 \times 28 \times 8 \rightarrow \text{Conv}(16) \rightarrow 14 \times 14 \times 16$

peak = 6 272 values after Conv1 → **25 kB float32 vs 6.3 kB int8**

Same model, same accuracy target — **4× RAM back**, just by changing the number format.

Why int8 is also *faster* (MACs $\neq$ latency)

MAC count doesn't change when you quantise. Latency drops anyway:

- **Memory traffic** — 4× fewer bytes through the bus and caches
- **SIMD** — Cortex-M4 DSP instructions chew multiple 8/16-bit lanes per cycle (CMSIS-NN kernels)
- typical TinyML latency window: 10–500 ms on Cortex-M4 — int8 commonly lands 2–4× lower than float

The FPU makes float *viable* on the M4F — int8 makes it *cheap*.

Quantisation mechanics in one slide

Map floats to 8-bit integers with an affine transform:

$$\text{real_value} \approx \text{scale} \times (\text{int8_value} - \text{zero_point})$$

- `scale`, `zero_point` chosen so the observed value range fits into $[-128, 127]$
- Weights: ranges known after training → quantise directly (often per-channel)
- **Activations: ranges depend on input data** → need *calibration* with representative samples
- Rounding error $\approx$ uniform noise — small for well-conditioned layers, ugly for outlier-heavy ones

Post-training quantisation (PTQ)

1. Train in float32 as usual
2. Run a few hundred **representative windows** through the model, record activation ranges
3. Convert weights + activations to int8 with the calibrated scales
 - No retraining, minutes of work
 - Typical hit on small models: **$\approx$ 0–2 % accuracy**
 - Edge Impulse does this automatically — the *Quantized (int8)* deployment option, calibrated on your dataset

Quantisation-aware training (QAT)

- Insert **fake-quantise** ops during training — forward pass sees int8 rounding, backward pass trains through it
- The network learns weights that are robust to quantisation noise
- Costs: a (re)training run + framework support (`tfmot` , EI enterprise)

	PTQ	QAT
Effort	minutes	retraining
Accuracy loss	small, sometimes ugly	usually negligible
When	always try first	only if PTQ hurts

Edge Impulse EON compiler

Two independent switches at deployment time:

	float32	int8 (quantised)
TFLite-Micro	baseline	smaller
EON compiled	smaller RAM/ROM	smallest

- EON = model graph compiled to direct C++ instead of interpreted FlatBuffer
- EI's published claim: same accuracy, up to ~25–55 % less RAM / ~35 % less flash vs TFLM
- Studio shows **estimated RAM / flash / latency per option** before you build — read them, then verify with `micros()`

Studio tells you the price up front

Deployment page, per option:

	Quantized (int8)	Unoptimized (float32)
Latency	~ __ ms	~ __ ms
RAM	~ __ kB	~ __ kB
Flash	~ __ kB	~ __ kB
Accuracy	__ % (model testing) __ %	

Estimates are for a reference M4 @ 80 MHz — *your* 64 MHz numbers come from `micros()` in the lab.



Lab part (a) starts now — EI int8 vs float32 (~15 min to queue)

[exercises/module9-optimisation/README.md](#) → part (a)

- Redeploy the Module 7 EI model **twice**: EON int8 and float32
- Note Studio's flash/RAM/latency estimates for both — the lab measures reality
- **Done when:** both builds are queued/downloaded

Quantising trees — emlearn dtypes

```
cmodel = emlearn.convert(clf, method='inline', dtype='int16_t')
cmodel.save(file='fan_model_i16.h', name='fan_model')
```

- Trees have **no weights** — only feature *thresholds*. `dtype` sets the type your features are compared in: `float`, `int32_t`, `int16_t`, `int8_t`
- Smaller dtype → smaller flash, cheaper compares — but your features must **fit the range** (scale them to integers first!)
- `int8` means 256 distinct feature levels — check your feature histograms before choosing
- Inference itself was already multiplication-free — trees start where NNs end up



Lab part (c) — emlearn dtype ladder (~10 min)

`exercises/module9-optimisation/emlearn_dtype_compare.py`

- Convert the Module 7 forest as float / int32 / int16 / int8
- Compare header size + prediction agreement vs float
- **Done when:** you know which dtype your fan forest can afford

Aside: AlfES Q7 (from the Fraunhofer workshop)

- AlfES supports **Q7** (int8 fixed-point) inference — workshop Task 2 ships the same network as `aifes_f32_fnn.h` and `aifes_q7_fnn.h`
- Benchmarks from the workshop deck: vs TFLite-Micro, AlfES measured ~2× faster and up to ~4× less memory on small FNNs
- The workshop projects already target nRF52840 (`nano33ble`) — they build for the RAK4631 with a one-line board change
- **Bonus lab:** run f32 vs Q7 side by side, `micros()` both

Pruning — most weights are freeloaders

Trained networks are over-parameterised: histogram the weights of your fan NN — a fat spike around zero.

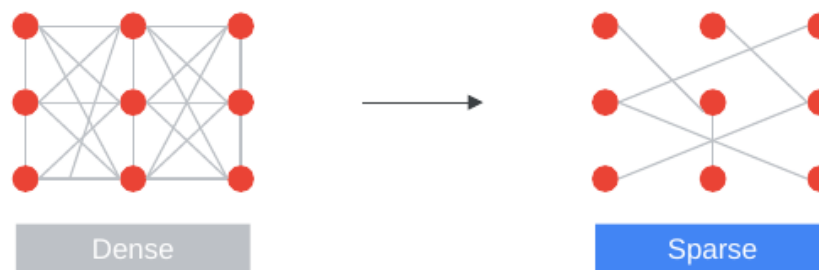


Figure: TinyML on edX (HarvardX), V.J. Reddi et al. — used with permission

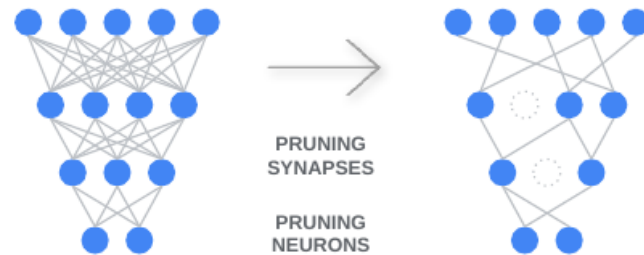
Magnitude pruning:

1. Rank weights by $|w|$
2. Zero out the smallest X %
3. Fine-tune the survivors (the network heals)
4. Repeat / schedule until target **sparsity** (e.g. 80 %)

Small dense models often tolerate **50–80 % sparsity at < 1 % accuracy loss**. Your notebook

Structured vs unstructured — in pictures

Unstructured Pruning



Structured Pruning

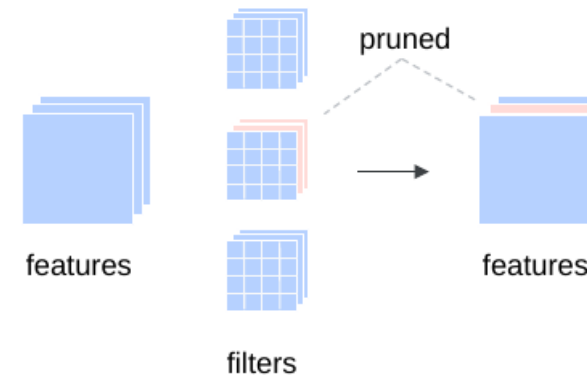


Figure: TinyML on edX (HarvardX), V.J. Reddi et al. — used with permission

Structured vs unstructured

	Unstructured	Structured
What's removed	individual weights	whole neurons / channels / filters
Sparsity pattern	scattered zeros	smaller dense tensors
Accuracy per removed param	better	worse
Speedup on MCU	none without sparse kernels	real — fewer MACs, less RAM
Size win	only after compression (gzip/OTA)	immediate

A dense matmul with 80 % zeros still executes 100 % of its MACs.

When pruning pays (and when it doesn't)

Pays:

- Compressed storage & OTA updates — sparse weights gzip beautifully
- Structured pruning of over-sized layers (our CNN's 560 → 8 dense layer is a prime target)
- Combined with quantisation: prune → quantise compounds

Doesn't:

- Unstructured pruning + standard TFLM/EON kernels → same latency, same RAM
- When the honest fix is a smaller architecture — *always try that first* (remember the grid search: 3 filters beat 8)

Pruning in Keras (tensorflow_model_optimization)

```
import tensorflow_model_optimization as tfmot # VERIFY: needs tf.keras (Keras 2);
        # with TF>=2.16 set TF_USE_LEGACY_KERAS=1
prune = tfmot.sparsity.keras.prune_low_magnitude
schedule = tfmot.sparsity.keras.PolynomialDecay(
    initial_sparsity=0.0, final_sparsity=0.8,
    begin_step=0, end_step=end_step)

pruned = prune(model, pruning_schedule=schedule)
pruned.compile(...)
pruned.fit(..., callbacks=[tfmot.sparsity.keras.UpdatePruningStep()])
final = tfmot.sparsity.keras.strip_pruning(pruned) # remove the wrappers
```

Notebook: sweep final_sparsity $\in \{0.5, 0.7, 0.8, 0.9\}$, plot **sparsity vs accuracy**, compare gzipped sizes.



Lab part (b) — prune the dense model (~25 min)

[exercises/module9-optimisation/notebooks/pruning_dense_model.ipynb](#)

- Magnitude-prune the Module 7 dense NN: sparsity 0.5 $\rightarrow$ 0.9
- Plot sparsity vs accuracy; gzip the models — watch compression grow
- **Done when:** you can say where YOUR model's accuracy cliff is

The optimisation ladder

1. **Right-size the architecture** — smallest model that meets accuracy (Module 7 grid search)
 2. **PTQ int8 + EON** — near-free 4× on weights & activations
 3. **Structured pruning / smaller layers** — if RAM or latency still hurts
 4. **QAT** — if and only if PTQ cost too much accuracy
 5. Exotic (int4, distillation, sparse kernels) — beyond this course
- Measure after every rung. `micros()` doesn't lie; estimates sometimes do.



Lab wrap — finish the measurements, fill the table

All three parts started at their checkpoints — now close them out:

1. **(a) int8 vs float32:** flash both builds, put `micros()` + real flash/RAM next to Studio's estimates
2. **(b) pruning:** finish the sparsity sweep, note YOUR accuracy cliff
3. **(c) dtypes:** pick the dtype your fan forest ships with

Done when: the optimisation table is complete + one sentence: *what would you ship, and why?*

Three days, one system — built by you

LIS3DH @ fan → your forwarder → labelled dataset (EI)
→ features (Python $\rightleftharpoons$ C, validated) → RF / NN / CNN (compared)
→ deployed on RAK4631 → anomaly-gated → quantised & measured

- Day 1: why TinyML, the hardware, first data
- Day 2: the fan rig, features, framework landscape
- Day 3: models in a shoot-out, unknown-fault detection, optimisation

You didn't follow a tutorial — you built the pipeline that tutorials are about.

From PC fan to production condition monitoring

Course rig	Production (pumps...)
120 mm fan, 5 fault states	real machines, unknown fault space → anomaly-first
RAK1904 on the frame	qualified industrial sensors, proper mounting
20-min datasets	months of fleet data, per-unit baselines, re-baselining after service
serial + LED	LoRa/BLE alerts (the SX1262 is already on your board), fleet dashboards
accuracy on a test split	false-alarm cost vs missed-fault cost, drift monitoring, model updates OTA

Same pipeline. Same maths. Bigger consequences.

Wrap-up

- All materials, code and solutions: course repo (decks + [exercises/](#))
- Keep the kit — the forwarder, features and models are yours to reuse
- Inspiration: previous student projects (knock detector, drone-sound detection, bike-theft alarm, ...)
- Questions / reach out: morten@ece.au.dk

Thanks — now go monitor some machines.